



Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Point of Contact:

Edward C. Zimmermann

E-mail: <ezimmermann@acm.org>

Abstract / Summary

*CoreQuarry is a hybrid knowledge retrieval engine which emerged in 2026 from Project Schmate (שמאטע) for **re-Isearch**. It unifies lexical, structural, and semantic search into a single, high-performance platform. Unlike existing vector databases or traditional search engines, it supports true positional indexing, structure-aware queries, and typed object retrieval, enabling precise and contextually-aware search over heterogeneous document corpora. By leveraging memory-mapped, append-only indexes and a two-tier address-based caching system, the engine achieves extremely low memory footprints while scaling to handle complex, hybrid RAG queries on consumer hardware, including laptops and edge devices.*

CoreQuarry Innovation:

- Rich Operator Support: Full set of binary and unary operators (AND, OR, XOR, NEAR, BEFORE/AFTER, PEER, distance-based, field-specific operations) allow precise relational queries beyond traditional BM25-based ranking.
- Hybrid & Extensible Indexing: Lexical, structural, numeric, geospatial, phonetic, and vector types are first-class citizens; plug-ins allow user-defined types and new algorithms without redesigning the core.
- Efficient Incremental Indexing: Append-only design with versioned updates, delete flags, and background garbage collection supports concurrent search and continuous ingestion, making it suitable for live AI pipelines.

General Impact:

This engine enables privacy-preserving, local-first AI retrieval, supporting applications in legal research, medical knowledge, industrial edge AI, and large-scale document analytics. Its small footprint and high flexibility make it uniquely suited for embedded devices, laptops, and offline environments, dramatically lowering the barrier for organizations to implement robust AI-powered search without relying on cloud services.

Key Product Differentiators

- Unified Hybrid Retrieval: Combines lexical, structural, and semantic (vector) search in a single engine.
- Rich Query Operators: Supports advanced positional, proximity, field-specific, and relational operators beyond traditional BM25.
- Typed Object Indexes: Indexes dates, numbers, hashes, geospatial, phonetic, and vectors as first-class data types.
- Append-Only, Memory-Mapped Architecture: Enables concurrent search and continuous ingestion with minimal RAM usage (lexical-only: min ~8MB).
- Extensible & Pluggable: Supports new document types, custom fields, and alternative retrieval algorithms (HNSW, NSG, etc.).
- Local-First Deployment: Runs efficiently on laptops and edge devices, enabling privacy-preserving AI search without cloud dependencies.
- High Throughput: Demonstrated thousands of QPS on consumer hardware for complex hybrid queries.
- Extremely efficient and fast HNSW implementation with support for quantization and pass-through.

The system solves three major problems that current RAG and search solutions don't:

1. Hybrid Retrieval at Scale
 - Lexical + structural + semantic in a single index

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

- True boolean/proximity operators and field-scoped searches
 - Optional vector embeddings for AI-enhanced relevance
2. Local-first & Lightweight
 - Runs fully on consumer hardware (Mac M1, laptops, edge devices)
 - Lexical-only mode: <10MB RAM
 - No dependency on heavy external services (no Elasticsearch, no FAISS clusters)
 3. Extensibility & Heterogeneous Data
 - Typed object indexes (dates, numbers, geospatial, phonetics, hashes, vectors)
 - Plug-ins for custom document formats and schemas
 - Supports structured traversal, wildcards, field/path scoping

Potential Markets / Applications

1. Local AI Assistants
 - Law firms, medical practitioners, researchers: local RAG over sensitive documents
2. Edge AI Devices
 - IoT devices, robotics, industrial sensors: offline retrieval of structured datasets
3. Enterprise Knowledge Infrastructure
 - Replace multi-system RAG stacks (vector DB + search engine + pipeline)
 - Offers privacy, low-latency retrieval, and structured queries
4. Archival & Compliance
 - Historical corpora, standards, legal/financial archives: structural search + temporal/versioning
5. Developer Tooling
 - Embed directly into applications like SQLite: “SQLite for AI retrieval”

Support:



Through the [NGI0 Discovery Fund](#), a fund established by [NLnet](#) with financial support from the European Commission's [Next Generation Internet](#) programme, under the aegis of [DG Communications Networks, Content and Technology](#) under grant agreement No [825322](#).

Through the [NGI0 Commons Fund](#), a fund established by [NLnet](#) with financial support from the European Commission's [Next Generation Internet](#) programme, under the aegis of [DG Communications Networks, Content and Technology](#) under grant agreement No [101135429](#). Additional funding is made available by the [Swiss State Secretariat for Education, Research and Innovation](#) (SERI).

Through a Grant from the Bundesministerium für Forschung, Technologie und Raumfahrt (Germany) GRANT_NUMBER: [01IS22S32](#) (exploring support of the IPFS and supporting remote indexing). Through a grant from the European Commission Coordination and Support Action (CSA) on ICT standardisation (extending support for additional post ISO-8601:2019 features).

Additional support was provided by a grant from OpenData CH/Mercator Foundation CH.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Appendix I: Technology Backgrounder / The 30 year old new kid

re-Isearch is the modern evolution of iSearch—the pioneering open-source engine of the mid-90s we co-developed with CNIDR. Originally developed to solve the "document type" problem of the early web, the technology was reborn through the support of NLNet as well as grants from BMFTR (German Federal Ministry for Research, Technology and Space), Open Data Switzerland and the Mercator Foundation. Today, we've moved beyond lexical indexing adding with Project Schmate vector indexes to become CoreQuarry. It combines thirty years of architectural expertise with cutting-edge hybrid search to help mine the core of human knowledge.

Our philosophical ancestor is early SGML retrieval systems (like OpenText which grew out of a University of Waterloo project from Timothy Bray, Frank Tompa, and Gaston Gonnet to digitize the Oxford English Dictionary) but designed to support beyond SGML multiple document formats and data types; and modernized for:

- embeddings
- local-first compute
- RAG

For vector search we use the The Hierarchical Navigable Small World (HNSW) algorithm. Introduced in 2016 by Yu. A. Malkov and D. A. Yashunin in their paper, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs". It is the evolutionary successor to the Navigable Small World (NSW) graph, combining it with the hierarchical structure of Skip Lists.

Both our algorithms for lexical and vector structures were designed to move beyond simple linear scans. HNSW achieves logarithmic complexity $O(\log N)$ for vectors, while PatArrays provide similar efficiency for finding any substring in a massive corpus without an inverted index.

Hardware has progressed dramatically since the mid-1990s when those SGML retrieval systems were deployed. Back then:

- high end servers (e.g. Sun Ultra Enterprise, IBM RS/6000 SP, HP 9000, DEC Alpha servers, and high-end Intel Pentium Pro servers): had 1-4 GB RAM (mainly EDO or perhaps early ECC DRAM). The memory bus speed was ~60-66 Mhz and the bandwidth was ~200-500 MB/s. Latency was ~60-80ns.
- High-end disk arrays: 50–500 GB total (e.g., an EMC Symmetrix could hold ~400 GB) where Individual SCSI drives: 2–18 GB per spindle and RAID controllers with cache: 32–256 MB write cache

The Numbers Side-by-Side

Metric	Mid-1990s Enterprise	Modern 2026 Workstation	Improvement
RAM capacity	1–4 GB	128 GB–2 TB	~100–500×
Memory bandwidth	~400 MB/s	~300 GB/s	~750×
Memory latency	~70 ns	~75 ns end-to-end	Roughly flat
Disk seq. read	10–15 MB/s	12,000–14,000 MB/s	~1,000×
Disk random IOPS	100–200	1,000,000–2,000,000	~10,000×
Disk seek/latency	8–12 ms	0.02–0.1 ms	~100–500×
Storage capacity	400 GB (array)	4–8 TB (single drive)	~10–20× per drive
CPU clock	100–500 MHz	4,000–6,000 MHz	~10–60×

A fully configured mid-1990s enterprise server with 2–4 GB RAM, a modest disk array (~100 GB), RAID, and software licenses: \$500,000 – \$3,000,000+ (in 2026 dollars that's roughly \$1M – \$6M) **versus** ~\$8,000–\$9,000 for a 2026 high-end workstation and even under \$2000 for a fast consumer desktop (64-core, 64GB DDR5-6000 RAM, 2TB SSD etc). Even the €475 (Netto) Apple M4 Mini (standard M4 chip) offers 38 TOPS, 120GB/s memory bandwidth, 16GB RAM and 255GB SDD (under 0.1 ms seek latency). The NVIDIA DGX Spark, powered by the GB10 Grace Blackwell SOC, delivers up to 1 PetaFLOP,

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

memory bandwidth of approximately 273 GB/s, 128 GB of unified LPDDR5x memory and 4GB SSD for 4.349,00 €

What This Means for Information Retrieval

The most dramatic difference isn't raw speed — it's random I/O latency and IOPS, which is exactly what database queries and information retrieval hammers hardest.

- A full-table scan that took 10 minutes on a 90s SCSI array takes seconds on NVMe
- A database query requiring 10,000 random disk reads: 90s machine takes 80–120 seconds just in I/O wait; modern NVMe does it in ~10–50 milliseconds
- The entire working dataset that required careful buffer pool management in the 90s (because RAM was scarce) now fits entirely in RAM on a modern system — eliminating disk I/O almost entirely for hot data
- Memory bandwidth improvements mean in-memory analytics (columnar stores, etc.) can process data ~750× faster even if latency is similar

The 90s DBAs spent enormous effort on I/O optimization— index tuning, careful buffer management, disk striping — precisely because every random read cost 10ms. On modern hardware, that same problem is largely dissolved. The bottleneck has moved entirely to CPU cache and memory access patterns.

What this means is that algorithms like the inverted index are obsolete! Their historical advantage (accepting their significant limitations) makes no sense in 2026.

The inverted index was an engineering response to hardware constraints, not an ideal data structure:

- Disk was slow and expensive— you couldn't afford to scan raw text
- RAM was tiny— you couldn't hold documents in memory
- IOPS were precious— every random read cost 10–12ms
- So you precomputed posting lists to minimize disk seeks at query time

The entire architecture mantra of Lucene, early Google, and most 1990s IR systems was essentially: *"disk is our enemy, precompute everything possible to avoid touching it."*

Patricia Trees and Pat Arrays Fell Out of Favor in the 1990s because of hardware:

- Pointer chasing is catastrophically cache-unfriendly— every node traversal is a potential cache miss
- On 1990s hardware, a cache miss meant going to main memory at 60–80ns, and with tiny caches (L1 was 8–16KB) misses were constant
- The inverted index, despite its bulk, had more predictable, sequential access patterns that could be optimized with careful disk layout
- Patricia trees also consumed significant RAM for pointers — precious in an era of \$50,000/GB RAM
- Worst case pointer density: a tree over a million terms might have millions of pointer pairs, each potentially a cache miss during traversal

So Patricia trees were theoretically elegant but practically punished by memory hierarchies of the era.

Cache hierarchies have transformed things

Cache	1995 Size	2026 Size	Improvement
L1	8–16 KB	32–64 KB	~4×
L2	256 KB (rare)	512 KB–1 MB per core	~4–8×
L3	Nonexistent/tiny	64–256 MB (Threadripper)	∞ → huge
RAM	256MB–4GB	128GB–2TB	~500×

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

A Patricia tree over a large IR vocabulary (~1–10 million terms) occupies perhaps 500MB–2GB of RAM. On a modern system this fits entirely in L3 cache on high-end CPUs, or at worst in RAM with DDR5's enormous bandwidth.

When the whole tree lives in L3, pointer chasing is no longer going to slow DRAM — it's hitting ~10 cycle L3 latency vs the ~200 cycle DRAM penalty that killed Patricia trees in the 1990s.

PAT Array Problem (1992)	2026 Solution
Index too large for RAM	128GB–2TB RAM — fits comfortably
$O(\log n)$ random accesses slow	DDR5 random access ~75ns, L3 ~10ns — negligible
Construction too slow	Modern $O(n)$ suffix array construction (SA-IS algorithm) + fast CPUs
Binary search cache-unfriendly	Entire index in L3/RAM — cache misses collapse
No word-level ranking (BM25 etc.)	Can layer neural/embedding ranking on top

Our indexing architecture builds on ideas related to Patricia trees and Pat arrays, but extends them to address the I/O and scalability constraints that have historically limited their use in large-scale information retrieval. A Patricia tree provides an efficient compressed trie structure for prefix searches, but it requires pointer-heavy navigation and does not map well to modern memory-mapped storage or high-throughput disk access. Pat arrays, by contrast, store lexicographically ordered suffix or term references in contiguous arrays, improving locality but traditionally requiring the underlying corpus text to be stored alongside the index and incurring expensive disk reads during lookup. Our approach separates corpus storage from the index while maintaining compact addresses.

Caching Strategy based on Word Length Distribution

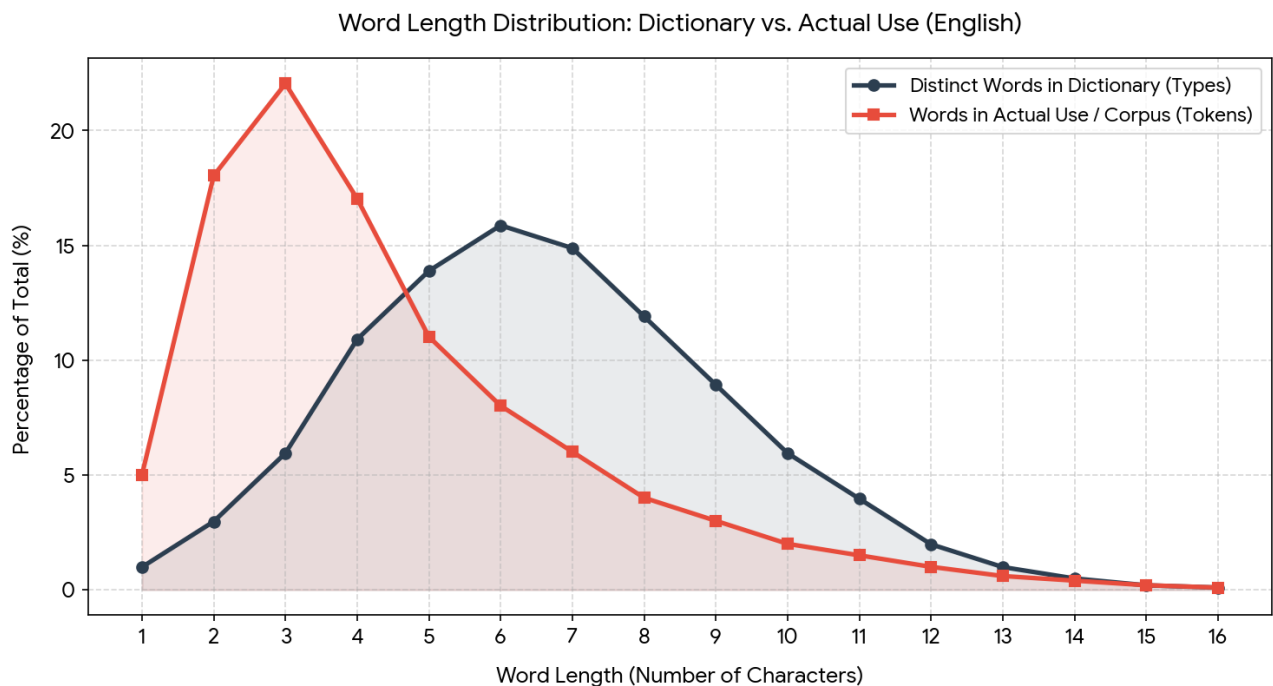
Information Efficiency: Human communication is optimized for speed. According to **Zipf's Law of Abbreviation**, the words we repeat most frequently are compressed by evolution into the shortest possible forms

Distinct Words (Types) follow a **unimodal distribution** (a hump-shaped curve). The vast majority of dictionary words are moderately long (typically 6–9 characters in English) because a language needs permutations of letters to create a unique vocabulary. Very short words are rare because there are only so many combinations you can make with 1 to 3 letters

Words in Use (Tokens) follow a **monotonic decrease or highly right-skewed distribution**. When analyzing a real book or corpus, massive frequency is dominated by short grammatical words ("the", "of", "and", "to"). This causes a huge spike at lengths 2–4, dropping sharply as words get longer.

Grammar vs. Content: Short words (2–4 letters) act as structural or grammatical "bridges" (e.g., pronouns, prepositions, articles). We reuse them constantly in every sentence, whereas long words (8+ letters) carry specific technical or descriptive content and are only used when strictly necessary

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.



Our 2-tier cache, originally introduced in our proprietary fork, allows the advantages of the separate corpus but removes the need to read it for words less than some pre-set prefix length.

This works remarkably well especially in English

- ➔ 1-7 letters: ~82.97%
- ➔ 8-10 letters: ~13.46%
- ➔ 11-14 letters: ~3.46%
- ➔ 15 letters: ~0.076%
- ➔ ≥16 letters: ~0.034% combined

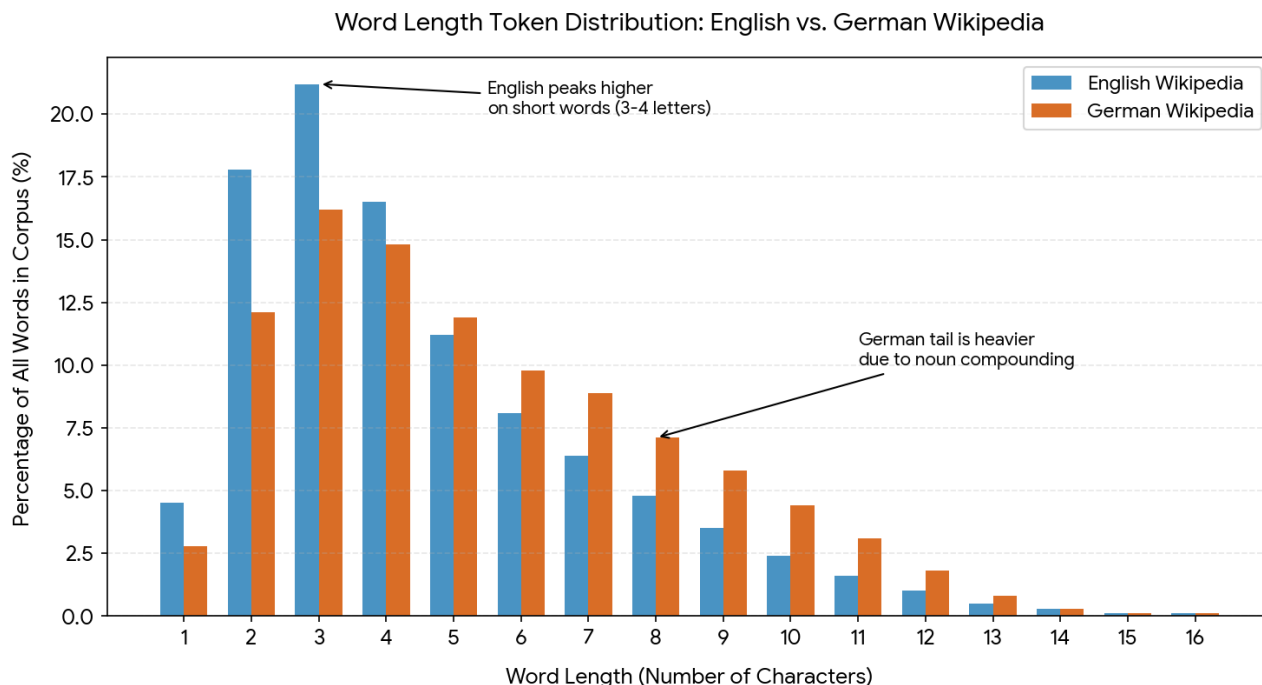
≈ **99.9% of words are under 15 characters long** in typical English text. **10 characters or less: ~96.4%**. A 10-character prefix is long enough that collisions are uncommon, so the resulting count will be close to the original dictionary size.

So even with a length set to 10, 96% or all words won't demand a corpus lookup. This cache is also relatively small since it would require only 26 bytes per unique word (for 64-bit indexes).

Source	Count
Words ≤10 chars	~118k
Unique prefixes from longer words	~44k
Total unique entries	~162k

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

≈150,000 – 170,000 max unique entries so we can safely assume that the maximum size of the cache will be under 4.5 MB.



German behaves very differently from English because of productive compounding (e.g., *Krankenhausverwaltungsgesetz*). So truncating to 10 characters collapses many more words.

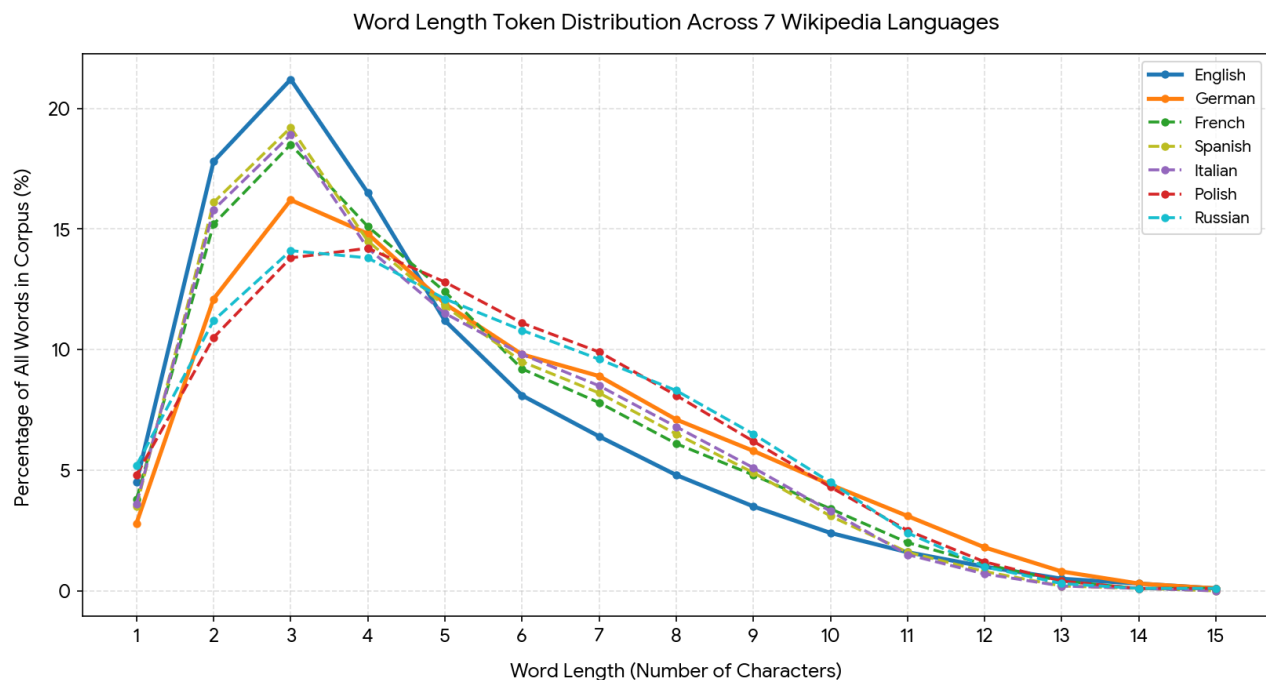
Length	Approx. share
≤10 letters	~40–50%
11–15 letters	~30–35%
16+ letters	~20–30%

Expanding this analysis across **three different language families** (Germanic, Romance, and Slavic) reveals clear patterns in how grammatical structures dictate token word length.

Comparative Insights by Language Family

- **The Romance Family (French, Spanish, Italian):** These three closely mirror each other. They behave like English with a strong peak around **3 to 4 characters**, heavily driven by shared grammatical features like short articles (*el, la, le, les, il, i*), prepositions (*de, di, en*), and conjunctions (*y, et, e*).
- **The Slavic Family (Polish, Russian):** Slavic languages completely lack definite and indefinite articles (no words for "a" or "the"). Because they skip these highly repetitive ultra-short words, their curves are much flatter and **shifted to the right**, peaking broader around **3 to 6 characters**. They also rely on extensive case-inflected suffixes, lengthening their baseline vocabulary.
- **The Germanic Family (German vs. English):** While English behaves like a Romance language in its word length due to heavy reliance on short particles, German stands out completely alone with its prolonged, thick tail spanning from **7 to 12+ characters** due to its compound word architecture.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.



Practical rule for optional prefix length according to language for 95% unique

Language	Typical lexicon size	Safe prefix length	Main reason
English	~170k	8–10	Short words, low inflection
German	~350k	12–14	Strong compounding
French	~200k	9–11	Shared Latin prefixes
Spanish	~200k	9–11	Verb inflection
Italian	~180k	9–11	Romance morphology
Russian	~200k	10–12	Prefix + case system
Swedish	~250k	11–13	Compounding
Greek	~200k	11–13	Rich inflection, long stems
Polish	~250k	12–14	Very rich case morphology

If one needs a prefix length that works well across all these languages:

Prefix length	Result
10 chars	Works well for English / Romance languages
12 chars	Good for most European languages
14 chars	Safe even for German, Polish, Swedish
✓ 12 chars	Best universal compromise: 12 characters

This usually keeps >90–95% of entries unique across European languages. In a mixed environment we can expect the cache to be under 8MB.

Memory Layout Optimization

- Phrase queries (term A followed by term B within distance d) become a merge of two contiguous array slices — elegant and cache-friendly
- Co-occurrence statistics can be computed with a single sequential pass
- Compression (delta encoding on doc_ids, positions) works extremely well on sorted runs — better than inverted index compression in many cases

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

- The whole structure is memory-mappable— OS handles paging, no custom buffer pool needed

Field/Path complexity for Lexical Search. Looking at TEI P5 guidelines as an extreme example with a very large vocabulary.

- ~600+ distinct elements (tags) in the full schema
- ~**250–300 attributes**
- Elements are grouped into modules (core, header, verse, drama, manuscript description, dictionaries, etc.)

However, no real TEI file uses all of them. No more than 100–200 distinct tags is common in a complex scholarly TEI document.

Typical ranges seen in TEI corpora:

Project complexity	Unique paths
Basic TEI text	100–300
Scholarly edition	300–800
Rich digital humanities corpus	800–1500
Large TEI aggregation	1500–3000+

So a single complex TEI document might easily have ~300–800 unique paths.

There are, of course, more complex document models than TEI.

- 1) JATS (Journal Article Tag Suite)
 - Used by PubMed Central.
 - Very deep citation, funding, and metadata structures.
 - Often 200+ tags per article.
- 2) HL7 CLINICAL DOCUMENT ARCHITECTURE

One of the most complex real-world XML standards.

 - massive schema
 - heavy metadata
 - controlled vocabularies
 - deep nesting

Often thousands of unique paths across medical documents.
- 3) LEGAL XML / AKOMA NTOSO

Extremely complex legislative documents:

 - hierarchical law structures
 - cross references
 - amendments
 - semantic tagging

With current hardware in 2026, fast binary search, in absolute worst case, over 800 memory mapped files is still relatively performant: $O(N \log(M))$ where N is the number of indexes (here 800) and M is the maximum number of items in each. But since the indexes are sorted it is more typically: $O(\log(NM))$

Most path searches will be into just a few fields (tags) or paths whence a fast binary search in just a few memory mapped and cached files. As noted above: DDR5 random access ~75ns, L3 ~10ns — negligible.

Key Considerations:

- Mapping requires system calls (open + mmap) to map a file into the process's virtual address space. If the file's metadata is already in the OS page cache, the open() call is exceptionally fast.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

- **High-Speed Storage (NVMe/Storage Class Memory):** The cost of the page fault is increasingly determined by the latency of the underlying SSD, which is extremely fast on modern systems.
- **64-Bit Addressing:** There is virtually no limit on mapping file sizes on 64-bit systems.
 - **Cached Performance:** If the file is already in the page cache, the mmap() overhead is minimal
- **VMA Limits:** While Linux has a default limit on the number of memory map areas a single process can have (vm.max_map_count, often 65,530) this is well below max fields/paths.
- **The Unix kernel's page cache manages memory-mapped files lazily.**
- **Metadata Efficiency:** The OS is very efficient at opening a few files as needed.
- **By mapping only the files (lazily on demand) as they are actually needed we improve, given how the kernel uses variations of LRU to manage the page cache, efficiency.**

The advantage of the this approach is that memory demands don't spiral out of control even with an "excessive" number of fields. Since CoreQuarry only maps (using a LRU heuristic) the fields being searched memory demands for searches is modest. During index the number of fields/paths has no impact on consumption since indexing buffers are pre-defined irrespective of their numbers—relevant only to the maximum size of an individual document.

Appendix II: What is different?

70% of Retrieval-Augmented Generation (RAG) systems fail in production¹. They mostly fail at retrieval, not generation. The main issues are 1) context mismatch (chunking) 2) semantic collapse² 3) drift. Handling semantic drift is a "missing link" for Retrieval-Augmented Generation (RAG) systems because it represents the silent degradation of retrieval accuracy over time, turning previously reliable AI systems into sources of outdated or conflicting information. As enterprise data evolves, the "meaning-space" of the vector database shifts, causing embeddings of new documents to diverge from old ones, or queries to no longer align with stored knowledge. When a RAG system is updated with new documents without re-embedding old data or retraining the retriever, it creates a "single vector index" with mixed, conflicting, and divergent semantic contexts, known as **Knowledge Drift**.

Unlike a traditional code error that breaks a system, drift causes the RAG system to continue operating, but with silently declining accuracy. This degrades trust as the system provides subtly outdated or irrelevant information.

To address these issues, DeepQuarry (R&D), implements:

- **Continuous Monitoring:** Actively monitoring for "Retrieval Misses" and using specialized metrics to detect drift.
- **Semantic Segmentation:** segmenting data into more coherent semantic clusters working backwards from detected drift
- **Incremental Updating:** Re-indexing or re-embedding data with updated models.
- **Hybrid Search:** Combining semantic vector search with keyword search to catch specific, updated terminology that embeddings might miss.
- **Semantic Chunking:** Breaking down documents based on logical structure (not arbitrary character counts) to maintain the semantic integrity of each segment. The basic concept is that the explicit and implicit structure of documents anchors context.
- **Semantic indexes by field and sharding:** this avoids the network getting too crowded. When everything is thrown into a single pile the evidence is that starting as early as with 10k documents things start to fall part. The architecture implemented in CoreQuarry vastly extends the document capacity.

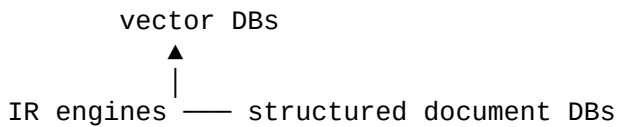
1 <https://python.plainenglish.io/why-70-of-rag-implementations-fail-and-the-6-things-that-separate-production-grade-systems-97501c11b682>

2 https://dho.stanford.edu/wp-content/uploads/Legal_RAG_Hallucinations.pdf

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Appendix III: Competitive Landscape

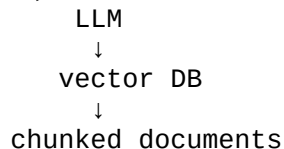
Our architecture uniquely overlaps three domains simultaneously:



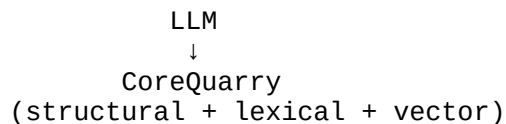
The engine has several advantages commercially:

- runs locally
- small footprint
- high performance
- hybrid retrieval
- structured document awareness

Right now, RAG infrastructure is mostly:



This architecture is deeply flawed. Instead or model:



If positioned correctly, it could disrupt the market become the retrieval layer for local AI systems.

1) PRIMARY DIRECT COMPETITORS (COMBINING LEXICAL + VECTOR SEARCH)

Elasticsearch

- Lucene-based
- supports BM25 + vector search
- widely used in RAG stacks

Comparative Weakness:

- poor structural awareness
- heavy infrastructure
- BM25-centric ranking
- Written in Java
- Triple license / AGPL

OpenSearch

Fork of Elasticsearch.

Weakness:

- same Lucene constraints

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Weaviate

- hybrid search
- object store
- semantic queries
- BSD 3-Clause license.

Weakness:

- vectors first
- weak lexical IR
- heavy infrastructure
- written in Go. This means some memory is not immediately available for reuse when it is no longer needed.

Qdrant

- fast ANN (HNSW)
- filtering
- Apache 2.0

Weakness:

- minimal lexical search
- no structural IR

Milvus

Heavily-modified forks of third-party open source [similarity search](#) libraries, such as FAISS, DiskANN and HNSWlib.

- large-scale vector DB (Kubernetes clusters)
- Apache 2.0

Weakness

- not really a document retrieval engine
-

2) CLASSICAL IR SYSTEMS

Apache Lucene

The foundation for:

- Elasticsearch
- Solr
- OpenSearch

Weakness:

- Java (written entirely in Java).
- inverted index constraints
- comparatively limited boolean model
- BM25-centric scoring
- limited structural awareness
- is resource hungry.
- Lucene writes new data to new, immutable segments, which are merged later. Merging is a resource-intensive, slow process.
- Deep paging limits: Standard paging is inefficient for deep searches, often limited to the first 10,000 documents to prevent cluster instability.

Apache Solr

Enterprise Lucene server.

Weakness

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

- heavy
- poor hybrid semantic search

Tantivy

Follows Lucene's fundamental architecture and algorithms, including BM25 scoring and similar indexing strategies, but takes advantage of Rust's memory safety and performance characteristics.

- Faster than Lucene
- MIT license

Weakness

- many like Lucene

Xapian

Interesting academically (long history) but not hybrid.

Weakness

- inverted index
- GPL license

3) VECTOR SEARCH ENGINES

FAISS

Very fast but:

- vectors only
- no document model

Weakness:

Slower than Quarry

ScaNN

Google ANN system.

HNSWlib

ANN library only. This is the starting code basis for the HNSW layer.

4) STRUCTURED DOCUMENT DATABASES

MarkLogic

This is probably the closest philosophical competitor.

Features

- XML aware
- structural queries
- strong IR

But:

- extremely heavy enterprise system
- not local-first
- not optimized for RAG

BaseX

XML structural search.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Weakness:

- no vector search
- limited IR capabilities

eXist-db

Similar category.

5) EMBEDDED DATABASES (INDIRECT COMPETITORS)

SQLite

Many RAG systems are now built on top of SQLite + vector extensions.

DuckDB

Often used locally for data analysis.

6) RAG FRAMEWORK ECOSYSTEMS

These are indirect competitors, because they often define the retrieval stack.

LangChain

Often integrates with:

- FAISS
 - Pinecone
 - Qdrant
-

LlamaIndex

Focused on document ingestion for RAG.

Appendix IV: Used Technologies and main AI tech

The core is a novel multimodal search and retrieval engine. It is a kind of hybrid between full-text, XML, object and graph noSQL-db that natively ingests a wide range of document types and data formats. Lexical search is based not on inverted indexes but on Pat Arrays with a two-tier address-based caching system that solved its main limitations. Dense vector search is handled by HNSW. The implementation is based on the HNSWlib references but heavily enhanced and turbo-charged.

Behind our embeddings is our fork of bert.cpp and the mainlined llama.cpp. Behind these is the GGML tensor library. Unlike mainstream vendor libraries (like PyTorch, TensorFlow, or TensorRT) built for data-center GPUs, GGML runs large transformer models efficiently on commodity hardware.

1. Superior CPU and Edge Device Execution.

Mainstream libraries rely heavily on massive GPU VRAM to hold model weights during inference. GGML is written in plain C and C++, making it highly portable.

- *Zero Dependencies:* It requires no heavy third-party stacks (such as CUDA or ROCm) just to run on a standard machine.
- *Hardware Interoperability:* It is optimized for x86 architectures (AVX, AVX2, AVX-512) and Apple Silicon (via native ARM NEON and Metal).
- *System RAM Utilization:* Because most modern consumer computers feature far more system RAM than GPU VRAM, GGML natively bridges the gap by enabling models to run efficiently on standard CPUs or via CPU-GPU offloading.

2. **Accelerators:** GGML features a highly modular backend architecture. This layout allows it to offload computation graph execution to a variety of hardware accelerators via specific hardware drivers, completely bypassing heavy Python stacks.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

3. **Advanced, Fine-Grained Quantization:** One of GGML's most compelling features is its pioneering approach to model quantization (compressing floating-point numbers into lower bit-widths, such as 4-bit or 5-bit).

- *K-Quants:* GGML utilizes highly engineered, variable bit-width quantization strategies (like Q4_K_M) that minimize the traditional loss in output accuracy.
- *Memory Compression:* It shrinks the memory footprint of massive models by up to 70-80% without heavily compromising the model's perplexity or reasoning capabilities.

4. **Streamlined Portability** (The GGUF Standard). GGML's associated file format, GGUF, was specifically designed to solve the fragmentation and loading issues of older ML formats.

5. **No Python Overhead**

6. **Specialized Text-Generation Ecosystem**

While mainstream libraries like TensorRT or PyTorch remain undisputed for model training and high-throughput server farms, GGML democratizes access to AI by allowing us to run powerful models locally on mainstream hardware.

While we build upon a heavily refactored bert.cpp we also support llama.cpp for more modern sophisticated models that don't use the BERT architecture such as EuroBERT.

Appendix V: Accelerators

Our lexical algorithms and HNSW implementation is, by design, CPU bound (the later using heavily optimized SIMD instructions) to leave the accelerators free to do the heavy lifting of running the models. Even on an Apple M1 (Pro) we've clocked 13k QPS-- on an M5 Pro we expect 30,000 to 39,000+ QPS-- so we found ample state-of-the-art performance already on CPUs.

1. NVIDIA CUDA

- *Quality:* This is the most mature, heavily optimized backend in the GGML ecosystem. It receives immediate day-one updates for new architectures.
- *Performance:* Offers the highest token-per-second generation rates. It handles massive context windows and batched requests effortlessly.
- *Efficiency:* While desktop NVIDIA cards draw significant wattage, the work-per-watt remains high because the GPU completes the generation sequence rapidly and returns to idle.

2. Apple Metal

- *Quality:* Flawless integration natively recognized by macOS and iOS.
- *Performance:* Extremely rapid. Because Apple Silicon uses a Unified Memory Architecture, the CPU and GPU share the same physical RAM pool. GGML leverages this via mmap, allowing a model to load instantly without a slow transfer over a PCIe bus.
- *Efficiency:* Unmatched. Users can run quantized 70B models locally on a MacBook with minimal heat production, preserving battery life during background operations.

3. Vulkan (The Compatibility Champion)

- *Quality:* Highly robust and serves as the best open-source, vendor-agnostic fallback.
- *Performance:* While it slightly lags behind native CUDA or ROCm tweaks, it delivers impressive matrix-multiplication speeds across diverse hardware, including Steam Decks, Raspberry Pis, and mixed-GPU setups.
- *Efficiency:* Balanced. Vulkan provides hardware acceleration on low-power devices that lack

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

dedicated AI chips.

Appendix VI: Vector search performance

Speed matters because RAG pipelines are dominated by retrieval latency. Faster retrieval can dramatically improve RAG pipelines

System	Approx QPS (768d)	Notes
Qdrant	~1k–3k	depends heavily on HNSW tuning
Milvus	~2k–5k	heavier infra
FAISS	~5k–10k	optimized C++ but limited system features
Elasticsearch	usually <2k	heavy overhead
Quarry HNSW	13k	C++.

Comparing to FAISS and single thread on an Apple M1:

Index type	Dataset size	QPS (1 thread)	Notes
Flat (exact search)	1M vectors	40–120 QPS	brute-force dot products
IVFFlat	1M vectors	150–400 QPS	typical ANN config
IVFPQ	1M vectors	300–900 QPS	quantized vectors
HNSW (FAISS)	1M vectors	200–600 QPS	depends heavily on efSearch
Quarry HNSW	1M vectors	3000 QPS	Faster for quantized vectors

The C++ HNSWlib was already one of the fastest HNSW implementations on the market. We kept to CPU only (to allow better utilization of the accelerators for the embedding models) but turbo-charged it to be 4-6x faster with Fp32 on ARM hardware and expanded its support for larger datasets through the reduction of memory demands through quantization. Comparing our optimized version with a straight compile (Clang without -O) the difference was 20x on an Apple M1. We also added support for pre-quantised models. These require not just much less memory but are also significantly faster than Fp32 models—since we implemented pass-through our HNSW graph networks also demand far less memory than solutions that pad to Fp32.